

Documentación operativa de DocLevel

Fecha de actualización: 26 de junio de 2026 Sitio web: <https://doclevelacademy.com/> Panel administrativo: <https://doclevelacademy.com/admin> Proyecto: DocLevel

1. Resumen ejecutivo

DocLevel es una página web de educación médica digital. Su función principal es mostrar cursos médicos, permitir que los visitantes exploren el catálogo y dar acceso a un panel administrativo desde donde el equipo puede crear, editar o eliminar cursos.

El proyecto está construido con Next.js, desplegado en Vercel y conectado a MongoDB Atlas. La misma aplicación maneja la parte visual del sitio y también los endpoints de API bajo la ruta **/api**.

Actualmente el sitio incluye:

- Página principal con curso destacado.
- Catálogo de cursos.
- Páginas individuales por curso.
- Landing de inscripción para el curso de “Las Primeras Horas de tu Bebé”.
- Página de contacto.
- Páginas legales: aviso legal, privacidad, cookies, términos y mapa del sitio.
- Panel administrativo protegido.
- API interna para cursos, categorías, contacto y autenticación.
- Base de datos MongoDB Atlas para cursos, administradores y mensajes de contacto.

2. Público objetivo de esta documentación

Este documento está pensado para:

- Compañeros de equipo que necesitan entender cómo funciona el proyecto.
- Socios de la empresa que quieren tener una visión clara del sitio.
- Personas técnicas que deberán mantener, desplegar o actualizar la plataforma.
- Personas no técnicas que necesitan saber qué partes existen y cómo se administran.

3. URLs importantes

Uso	URL
Sitio público principal	https://doclevelacademy.com/
Panel administrativo	https://doclevelacademy.com/admin
Catálogo de cursos	https://doclevelacademy.com/courses

Uso	URL
Contacto	https://doclevelacademy.com/contact
API de cursos	https://doclevelacademy.com/api/courses
API de estado	https://doclevelacademy.com/api/health

Nota: el dominio también puede resolver con **www**, por ejemplo **<https://www.doclevelacademy.com/>**.

4. Tecnologías usadas

Área	Tecnología
Frontend	Next.js 14, React 18
Backend/API	API Routes de Next.js
Base de datos	MongoDB Atlas
Autenticación	JWT + bcrypt
Hosting	Vercel
Estilos	Tailwind CSS
Componentes UI	Radix UI + componentes propios
Scripts de mantenimiento	Node.js y Python

5. Estructura general del proyecto

```

DocLevel/
├── app/                Páginas y rutas API de Next.js
│   ├── page.js        Página principal
│   ├── admin/page.js  Panel administrativo
│   ├── courses/page.js Catálogo de cursos
│   ├── courses/[id]/page.js Página individual de curso
│   ├── contact/page.js Página de contacto
│   └── api/[...path]/route.js API interna
├── components/        Componentes reutilizables
├── lib/               Conexión MongoDB, auth, catálogo y utilidades
├── public/            Imágenes públicas y assets de marca
├── scripts/           Scripts de administración y diagnóstico
├── docs/              Documentación del proyecto
├── middleware.js       Protección de /admin con Basic Auth
├── next.config.js      Configuración Next.js
├── vercel.json         Rewrites para landings
├── package.json        Dependencias y comandos
└── .env.local          Variables locales sensibles

```

6. Cómo funciona la página web

6.1 Página principal

Archivo principal: **app/page.js**

La página principal carga cursos desde MongoDB. Si por alguna razón MongoDB no responde, usa un catálogo de respaldo definido en **lib/doclevelCourses.json** y normalizado por **lib/courseCatalog.js**.

La página muestra:

- Navbar con logo, cursos, contacto y acceso a admin.
- Hero principal con el curso destacado.
- Botón para ver el curso.
- Botón para explorar el catálogo.
- Cursos agrupados por categoría.
- Footer con enlaces legales.

6.2 Catálogo de cursos

Archivo principal: **app/courses/page.js**

El catálogo permite:

- Ver todos los cursos.
- Buscar por título, descripción o especialidad.
- Filtrar por categoría.
- Ver cursos disponibles o marcados como próximamente.

Categorías actuales permitidas:

- Pediatría
- Odontología
- Ginecología
- Cardiología

Estas categorías están definidas en **lib/courseCategories.js**.

6.3 Página individual de curso

Archivo principal: **app/courses/[id]/page.js**

Cada curso tiene una página con:

- Imagen/banner.
- Título.
- Descripción.
- Especialidad.
- Estado.
- Experto.
- Duración.
- Precio.
- Contenido adicional.

- Cursos relacionados.
- Botón de inscripción o enlace a landing.

Si el curso está marcado como **coming_soon**, la página muestra estado de “Próximamente”.

6.4 Página de contacto

Archivo principal: **app/contact/page.js**

Permite enviar:

- Nombre.
- Email.
- Mensaje.

Los mensajes se guardan en la colección **contacts** de MongoDB.

6.5 Páginas legales

Existen páginas para:

- Aviso legal.
- Política de privacidad.
- Política de cookies.
- Términos.
- Mapa del sitio.

Estas páginas ayudan a dar estructura formal y confianza al sitio.

7. Panel administrativo

URL: <https://doclevelacademy.com/admin> Archivo principal: **app/admin/page.js**

El panel administrativo sirve para gestionar cursos.

7.1 Protección de acceso

Actualmente el acceso al panel tiene dos capas:

1. ****Basic Auth del navegador****
 - Se controla desde **middleware.js**.
 - Usa las variables **ADMIN_EMAIL** y **ADMIN_PASSWORD**.
 - Es la ventana emergente del navegador que pide usuario y contraseña antes de abrir **/admin**.
2. ****Login interno del panel****
 - Se controla desde **app/api/[...path]/route.js**.
 - Valida el administrador guardado en MongoDB, colección **admins**.
 - Usa **bcrypt** para comparar la contraseña.
 - Si el login es correcto, genera un **JWT**.
 - El token se guarda en **localStorage** como **doclevel_token**.

Importante: si una persona tiene problemas para entrar a **/admin**, debe revisar si está fallando en la primera capa, Basic Auth, o en la segunda capa, login interno.

7.2 Funciones del panel

Desde el panel se puede:

- Ver la lista de cursos.
- Crear un curso nuevo.
- Editar un curso existente.
- Eliminar un curso.
- Marcar un curso como destacado.
- Cambiar estado entre disponible y próximamente.
- Agregar URL de video.
- Agregar URL de landing o registro.
- Agregar URL de banner.
- Agregar contenido adicional.

7.3 Campos de un curso

Campo	Uso
title	Nombre del curso
description	Descripción corta
category	Especialidad
video_url	Video opcional
landing_url	URL de inscripción o landing
banner_url	Imagen principal del curso
content	Contenido adicional
status	available o coming_soon
featured	Define si aparece como destacado
expert	Instructor o experto
duration	Duración o fecha
price	Precio

8. API interna

Archivo principal: **app/api/[...path]/route.js**

La API vive dentro del mismo proyecto Next.js.

8.1 Endpoints principales

Método	Ruta	Función
GET	<code>/api/health</code>	Revisa si la API responde
GET	<code>/api/categories</code>	Lista categorías disponibles
GET	<code>/api/courses</code>	Lista cursos
GET	<code>/api/courses/:id</code>	Obtiene un curso
POST	<code>/api/auth/login</code>	Login de administrador
GET	<code>/api/auth/me</code>	Valida token JWT
POST	<code>/api/contact</code>	Guarda mensaje de contacto
POST	<code>/api/courses</code>	Crea curso, requiere admin
PUT	<code>/api/courses/:id</code>	Edita curso, requiere admin
DELETE	<code>/api/courses/:id</code>	Elimina curso, requiere admin
POST	<code>/api/seed</code>	Inserta cursos si la base está vacía

8.2 Catálogo de respaldo

Si MongoDB falla, algunas partes del sitio usan cursos de respaldo desde:

```
lib/docLevelCourses.json
```

Esto evita que la página pública quede totalmente rota si existe un problema temporal de conexión con la base de datos.

Cuando la API responde con **"source": "fallback"**, significa que se está usando el respaldo y no la base real.

9. Base de datos MongoDB

La conexión se configura en:

```
lib/mongodb.js
```

Variables usadas:

- **MONGO_URL**
- **MONGODB_URI**, alternativa compatible
- **DB_NAME**

Si **DB_NAME** no existe, el proyecto usa por defecto:

```
DocLevel
```

9.1 Colecciones principales

Colección	Descripción
courses	Cursos publicados o próximos
admins	Administradores del panel
contacts	Mensajes enviados desde contacto

9.2 Administradores

Los administradores se guardan en la colección **admins**.

La contraseña no debe guardarse en texto plano. El proyecto usa bcrypt para guardar un hash seguro.

Para cambiar el administrador se usa:

```
python .\scripts\change_admin.py
```

Ese script:

- Lee **.env.local**.
- Se conecta a MongoDB.
- Pide correo actual.
- Pide correo nuevo.
- Pide contraseña nueva.
- Genera hash bcrypt.
- Actualiza el administrador.
- Elimina administradores anteriores.
- No muestra ni guarda la contraseña en texto plano.

10. Variables de entorno

Archivo local:

```
.env.local
```

Variables de producción:

Vercel → Project Settings → Environment Variables

10.1 Variables importantes

Variable	Dónde se usa	Descripción
MONGO_URL	Local y Vercel	Cadena de conexión MongoDB Atlas
MONGODB_URI	Local y Vercel	Alternativa compatible para conexión MongoDB

Variable	Dónde se usa	Descripción
DB_NAME	Local y Vercel	Nombre de la base de datos
JWT_SECRET	Local y Vercel	Firma de tokens JWT
SETUP_TOKEN	Local y Vercel	Token para proteger /api/seed en producción
ADMIN_EMAIL	Local y Vercel	Usuario para Basic Auth de /admin
ADMIN_PASSWORD	Local y Vercel	Contraseña para Basic Auth de /admin
ADMIN_PASSWORD_HASH	Opcional en Vercel	Hash para login alternativo por variable de entorno

10.2 Recomendación de seguridad

No se recomienda incluir contraseñas reales dentro de esta documentación si se va a compartir por correo, WhatsApp, PDF abierto o drive compartido.

La forma recomendada es:

- Documento principal sin contraseñas.
- Anexo de accesos separado.
- Contraseñas guardadas en un gestor seguro, por ejemplo 1Password, Bitwarden, Keeper o similar.
- Acceso solo para personas autorizadas.
- Rotación de contraseñas cuando alguien deja el equipo.

11. Accesos y credenciales

11.1 Acceso público

El sitio público no requiere usuario ni contraseña:

<https://doclevelacademy.com/>

11.2 Acceso al panel admin

Ruta:

<https://doclevelacademy.com/admin>

Flujo esperado:

1. Abrir **/admin**.
2. El navegador pide usuario y contraseña de Basic Auth.
3. Ingresar credenciales autorizadas.
4. Aparece el login interno del panel.
5. Ingresar correo de administrador y contraseña del panel.

6. Si todo es correcto, se muestra el panel de cursos.

Correo administrativo actual:

`admin@doclevelacademy.com`

La contraseña debe compartirse por un canal seguro, no dentro del PDF general.

Para completar accesos privados, usar el archivo:

`docs/ANEXO_ACCESOS_SEGURO.md`

12. Scripts de mantenimiento

Los scripts están en:

`scripts/`

12.1 Crear o actualizar admin desde Node.js

```
npm run admin:upsert
```

Usa:

- **ADMIN_EMAIL**
- **ADMIN_PASSWORD**
- **MONGO_URL** o **MONGODB_URI**
- **DB_NAME**

Este script crea o actualiza un administrador, pero no elimina otros administradores anteriores.

12.2 Cambiar admin y eliminar anteriores

```
python .\scripts\change_admin.py
```

Este es el script recomendado cuando se quiere dejar un solo administrador activo.

12.3 Reemplazar cursos base

```
npm run courses:replace
```

Reemplaza la colección **courses** con los cursos definidos en:

`lib/doclevelCourses.json`

Advertencia: este comando elimina los cursos existentes y los reemplaza.

12.4 Diagnóstico de MongoDB

```
npm run mongodb:diagnose
```

Sirve para revisar conexión con MongoDB Atlas.

13. Cómo correr el proyecto localmente

13.1 Requisitos

- Node.js instalado.
- npm instalado.
- Acceso al repositorio.
- Archivo **.env.local** configurado.
- Acceso a MongoDB Atlas.

13.2 Pasos

Abrir PowerShell o terminal de VS Code dentro de:

```
C:\Users\cgcon\OneDrive\Escritorio\DocLevel
```

Instalar dependencias:

```
npm install
```

Levantar proyecto:

```
npm run dev
```

Abrir:

```
http://localhost:3000/
```

Panel local:

```
http://localhost:3000/admin
```

Si el puerto 3000 está ocupado:

```
npx next dev --hostname 127.0.0.1 --port 3002
```

Luego abrir:

```
http://127.0.0.1:3002/
```

14. Deploy en producción

El sitio está desplegado en Vercel.

14.1 Variables necesarias en Vercel

En Vercel deben existir, como mínimo:

- **MONGO_URL**
- **MONGODB_URI**

- **DB_NAME**
- **JWT_SECRET**
- **SETUP_TOKEN**
- **ADMIN_EMAIL**
- **ADMIN_PASSWORD**

Nota: aunque el login interno use MongoDB, el archivo **middleware.js** protege **/admin** con Basic Auth, por eso **ADMIN_EMAIL** y **ADMIN_PASSWORD** siguen siendo importantes mientras ese middleware exista.

14.2 Después de cambiar variables

Cuando se cambia una variable en Vercel:

1. Guardar la variable.
2. Hacer redeploy del proyecto.
3. Probar producción.

Comandos útiles:

```
npm exec vercel -- env ls
npm exec vercel -- deploy --prod
```

15. Actualizaciones recientes importantes

15.1 Administrador actualizado

Se ejecutó el script Python para actualizar el administrador.

Resultado esperado:

- Administrador activo: **admin@doclevelacademy.com**
- Contraseña reemplazada.
- Administradores anteriores eliminados.
- Contraseña no mostrada por el script.

15.2 JWT_SECRET configurado

Se agregó **JWT_SECRET**, necesario para que el login admin funcione.

Si falta esta variable, aparece el error:

```
Missing JWT_SECRET environment variable
```

15.3 MongoDB restaurado en producción

Se verificó que **/api/courses** responde desde MongoDB y no desde fallback.

Señal correcta:

- La API devuelve cursos.
- No aparece **"source": "fallback"**.

15.4 Ajuste DNS para desarrollo local

Se ajustó la resolución DNS local en:

- `lib/mongodb.js`
- `scripts/diagnose-mongodb.js`
- `scripts/change_admin.py`

Esto ayuda cuando el equipo local o el proveedor de internet bloquea o falla con consultas SRV de MongoDB Atlas.

16. Flujo recomendado para editar cursos

1. Entrar a <https://doclevelacademy.com/admin>.
2. Pasar Basic Auth.
3. Iniciar sesión en el panel.
4. Revisar cursos existentes.
5. Para crear un curso:
 - Click en “Nuevo curso”.
 - Completar título, descripción, categoría y banner.
 - Elegir estado.
 - Agregar landing si aplica.
 - Guardar.
6. Para editar un curso:
 - Click en ícono de lápiz.
 - Modificar campos.
 - Guardar cambios.
7. Para eliminar:
 - Click en ícono de basura.
 - Confirmar eliminación.

Recomendación: antes de eliminar cursos reales, confirmar con el equipo comercial o académico.

17. Checklist para confirmar que todo funciona

Sitio público

- Abre <https://doclevelacademy.com/>.
- Carga el logo.
- Carga el curso destacado.
- El botón “Explorar catálogo” funciona.
- El catálogo abre correctamente.

- La página de contacto abre correctamente.

API

- <https://doclevelacademy.com/api/health> responde.
- <https://doclevelacademy.com/api/courses> devuelve cursos.
- La respuesta no muestra **"source": "fallback"** cuando MongoDB está funcionando.

Admin

- **/admin** pide Basic Auth.
- Basic Auth acepta credenciales autorizadas.
- El login interno acepta el administrador activo.
- Se cargan cursos en el panel.
- Se puede editar un curso de prueba.
- Se puede cerrar sesión.

Vercel

- Variables de entorno configuradas.
- Último deploy en estado "Ready".
- Dominio conectado correctamente.

MongoDB

- Cluster activo.
- Usuario de base de datos vigente.
- IP Access List permite conexiones necesarias.
- Colecciones **courses**, **admins** y **contacts** disponibles.

18. Capturas recomendadas para el PDF

Para que el documento sea fácil de entender, conviene agregar capturas en estas secciones:

1. ****Página principal****
 - Captura del hero con el curso destacado.
 - Ubicación sugerida: después de la sección 6.1.
2. ****Catálogo de cursos****
 - Captura de **/courses** mostrando filtros y tarjetas.
 - Ubicación sugerida: después de la sección 6.2.
3. ****Página individual del curso****
 - Captura de un curso abierto.
 - Ubicación sugerida: después de la sección 6.3.
4. ****Pantalla de Basic Auth****

- Captura de la ventana de usuario/contraseña del navegador.
- Ubicación sugerida: sección 7.1.
- No mostrar contraseñas.

5. ****Login interno del panel****

- Captura del formulario de login.
- Ubicación sugerida: sección 7.1.
- No mostrar contraseñas.

6. ****Panel de cursos****

- Captura de la lista de cursos.
- Ubicación sugerida: sección 7.2.

7. ****Formulario de crear/editar curso****

- Captura del modal de edición.
- Ubicación sugerida: sección 16.

8. ****Vercel****

- Captura de variables de entorno, ocultando valores.
- Captura del deploy en estado Ready.
- Ubicación sugerida: sección 14.

9. ****MongoDB Atlas****

- Captura del cluster y colecciones.
- Ocultar connection string y usuarios.
- Ubicación sugerida: sección 9.

19. Recomendaciones de seguridad

- No usar la misma contraseña para MongoDB y para el panel admin.
- No pegar contraseñas en chats, PDFs o correos.
- Rotar credenciales si alguien externo tuvo acceso.
- Mantener **.env.local** fuera de Git.
- Usar contraseñas largas y únicas.
- Revisar periódicamente quién tiene acceso a Vercel y MongoDB.
- Si se comparte este documento con socios, compartir la versión sin anexo de accesos.
- Si se comparte el anexo de accesos, hacerlo por canal seguro.

20. Próximas mejoras recomendadas

Estas mejoras no son obligatorias, pero harían el proyecto más seguro y mantenible:

1. Cambiar Basic Auth por un flujo único de login admin.

2. Evitar **ADMIN_PASSWORD** en texto plano para producción.
3. Usar un gestor formal de secretos.
4. Agregar respaldo automático de MongoDB.
5. Agregar roles de usuario si más personas administrarán cursos.
6. Agregar auditoría de cambios en cursos.
7. Agregar subida de imágenes desde el panel, en lugar de pegar URLs.
8. Agregar pruebas automáticas para API y panel admin.
9. Documentar proceso comercial de publicación de cursos.
10. Crear guía de estilo para banners, textos y precios.

21. Glosario rápido

Término	Significado
Next.js	Framework usado para construir la web
Vercel	Plataforma donde está publicado el sitio
MongoDB Atlas	Base de datos en la nube
JWT	Token usado para mantener sesión admin
bcrypt	Sistema para guardar contraseñas de forma segura
Basic Auth	Ventana simple del navegador que pide usuario/contraseña
Fallback	Datos de respaldo cuando MongoDB falla
Deploy	Publicación de una nueva versión
Environment Variables	Variables secretas/configuración del servidor

22. Contacto interno sugerido

Completar con responsables reales:

Área	Responsable	Contacto
Dirección general	Pendiente	Pendiente
Administración web	Pendiente	Pendiente
Desarrollo técnico	Pendiente	Pendiente
MongoDB/Vercel	Pendiente	Pendiente
Contenido académico	Pendiente	Pendiente